NLP Preprocessing and Text Classification

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand and process human language. With the rapid growth of digital platforms, a huge amount of textual data is generated daily, making it essential to analyze and extract meaningful information from text.

One of the important applications of NLP is text classification, where text is categorized into predefined classes. Fake news detection is a major real-world problem, as the spread of misinformation can create confusion and impact society negatively.

In this project, NLP techniques are used to classify news articles as fake or real. The text data is preprocessed using methods such as tokenization, stopword removal, stemming, and lemmatization. The processed text is then converted into numerical form using CountVectorizer and TF-IDF techniques. A machine learning model, Multinomial Naive Bayes, is used to perform the classification, and its performance is evaluated using different metrics.

Objective

The main objective of this project is to apply Natural Language Processing techniques and build a machine learning model for text classification. The goals of the project are:

1. To preprocess text data using tokenization, stopword removal, stemming, and lemmatization.
2. To convert text data into numerical form using CountVectorizer and TF-IDF.
3. To build a classification model using Multinomial Naive Bayes.
4. To evaluate the performance of the model using accuracy, precision, recall, and F1-score.
5. To visualize the results using graphs for better understanding.

Problem Statement

In today's digital world, a large amount of news is shared online, and it is difficult to verify whether the information is true or false. The spread of fake news can mislead

people and create serious social issues.

The problem is to develop a system that can automatically classify news articles as fake or real based on their textual content. This requires processing raw text data, extracting meaningful features, and applying machine learning techniques to accurately predict the category of the news.

Implementation

The implementation of the project is carried out step by step using Python. The following sections describe each stage of the process, starting from importing the required libraries to building and evaluating the model.

1. Importing Libraries

In this step, all required libraries are imported. Pandas is used for handling datasets, NumPy is used for numerical operations, and NLTK is used for natural language processing tasks such as stopword removal, stemming, and lemmatization. Scikit-learn is used for building the machine learning model and evaluating its performance.

```
import pandas as pd
import numpy as np
import nltk
import re

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
```

2. Download NLTK Data

NLTK provides predefined datasets such as stopwords and wordnet. Stopwords are commonly used words that do not carry significant meaning, and wordnet is used for lemmatization. These resources are downloaded to enable preprocessing.

```
nltk.download('stopwords')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

3. Load Dataset

The dataset consists of two files containing fake and real news articles. These files are loaded into dataframes using pandas so that they can be processed and analyzed easily.

```
fake = pd.read_csv('Fake.csv')
true = pd.read_csv('True.csv')
```

4. Add Labels

```
fake['label'] = 0   # Fake news
true['label'] = 1   # Real news
```

A new column called 'label' is added to both datasets. Fake news is assigned label 0 and real news is assigned label 1. This helps the machine learning model distinguish between the two categories.

5. Combine Dataset

Both datasets are combined into a single dataframe using concatenation. The data is then shuffled randomly to ensure that the model does not learn any pattern based on the order of the data.

```
df = pd.concat([fake, true])
df = df.sample(frac=1).reset_index(drop=True)  # shuffle
```

6. Create Content Column

The title and text columns are combined into a single column called 'content'. This provides complete information to the model, improving its ability to classify the news accurately.

```
df['content'] = df['title'] + " " + df['text']
```

7. Preprocessing Setup

In this step, tools required for preprocessing are initialized. Stopwords are loaded to remove unnecessary words, and stemmer and lemmatizer are initialized to reduce words to their base form.

```
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
```

8. Preprocessing Function

This function performs multiple preprocessing steps. The text is converted to lowercase, and special characters are removed using regular expressions. Tokenization is performed by splitting the text into words. Stopwords are removed to eliminate unnecessary words. Finally, stemming and lemmatization are applied to reduce words to their base forms.

```
def preprocess(text):
    text = text.lower()
    text = re.sub('[^a-z]', ' ', text)

    words = text.split()  # Tokenization
    words = [w for w in words if w not in stop_words]  # Stopwords

    stemmed = [stemmer.stem(w) for w in words]
    lemmatized = [lemmatizer.lemmatize(w) for w in words]

    return " ".join(stemmed), " ".join(lemmatized)
```

9. Apply Preprocessing

The preprocessing function is applied to all rows in the dataset. This creates two new columns containing cleaned text using stemming and lemmatization.

```
df[['stemmed_text','lemmatized_text']] = df['content'].apply(lambda x: pd.Serie
```

10. Train-Test Split

The dataset is divided into training and testing sets. The training data is used to train the model, while the testing data is used to evaluate its performance on unseen data.

```
X_train, X_test, y_train, y_test = train_test_split(df['stemmed_text'], df['lab
```

11. CountVectorizer

CountVectorizer converts text into numerical form by counting how many times each word appears. This creates a matrix representation of the text data.

```
cv = CountVectorizer(max_features=5000)

X_train_cv = cv.fit_transform(X_train)
X_test_cv = cv.transform(X_test)
```

12. TF-IDF

TF-IDF assigns weights to words based on their importance. It reduces the impact of frequently occurring words and highlights rare but meaningful words. This improves model performance.

```
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2))

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

13. Model Initialization

Multinomial Naive Bayes is chosen as the classification model. It is suitable for text data and works efficiently with word frequency features.

```
model_cv = MultinomialNB()
model_tfidf = MultinomialNB()
```

14. Training & Prediction

```
model_cv.fit(X_train_cv, y_train)
pred_cv = model_cv.predict(X_test_cv)
```

```
model_tfidf.fit(X_train_tfidf, y_train)
pred_tfidf = model_tfidf.predict(X_test_tfidf)
```

The model is trained using the training dataset. After training, it is used to predict labels for the test dataset.

15. Evaluation

Model performance is evaluated using accuracy and classification report. Accuracy shows how many predictions are correct, while precision, recall, and F1-score provide detailed evaluation.

```
acc_cv = accuracy_score(y_test, pred_cv)
acc_tfidf = accuracy_score(y_test, pred_tfidf)

print("CountVectorizer Accuracy:", acc_cv)
print("TF-IDF Accuracy:", acc_tfidf)

print("\nClassification Report (TF-IDF):\n")
print(classification_report(y_test, pred_tfidf))
```

```
CountVectorizer Accuracy: 0.9477728285077951
TF-IDF Accuracy: 0.9454342984409799

Classification Report (TF-IDF):

              precision    recall  f1-score   support

           0       0.95      0.95      0.95      4697
           1       0.95      0.94      0.94      4283

    accuracy                           0.95      8980
   macro avg       0.95      0.95      0.95      8980
weighted avg       0.95      0.95      0.95      8980
```

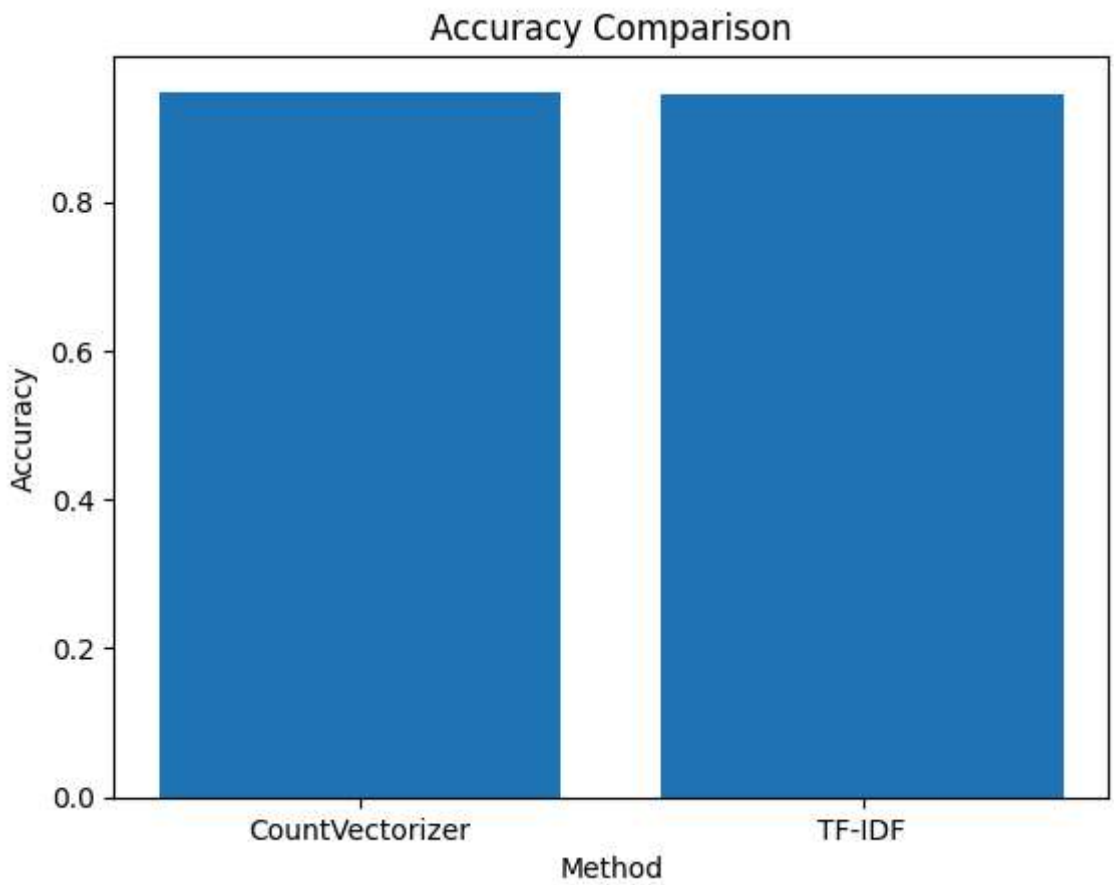
16. Results

```
import matplotlib.pyplot as plt

methods = ['CountVectorizer', 'TF-IDF']
accuracy = [acc_cv, acc_tfidf]

plt.figure()
plt.bar(methods, accuracy)
plt.title("Accuracy Comparison")
plt.xlabel("Method")
```

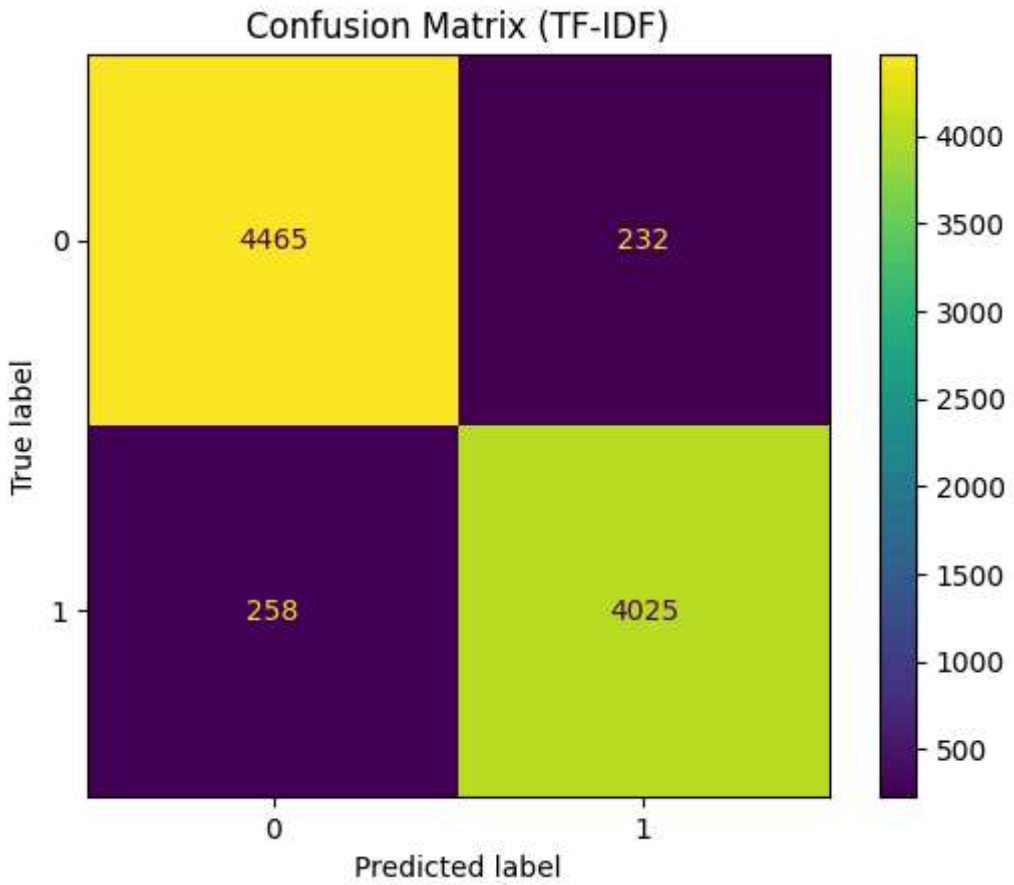
```
plt.ylabel("Accuracy")
plt.show()
```



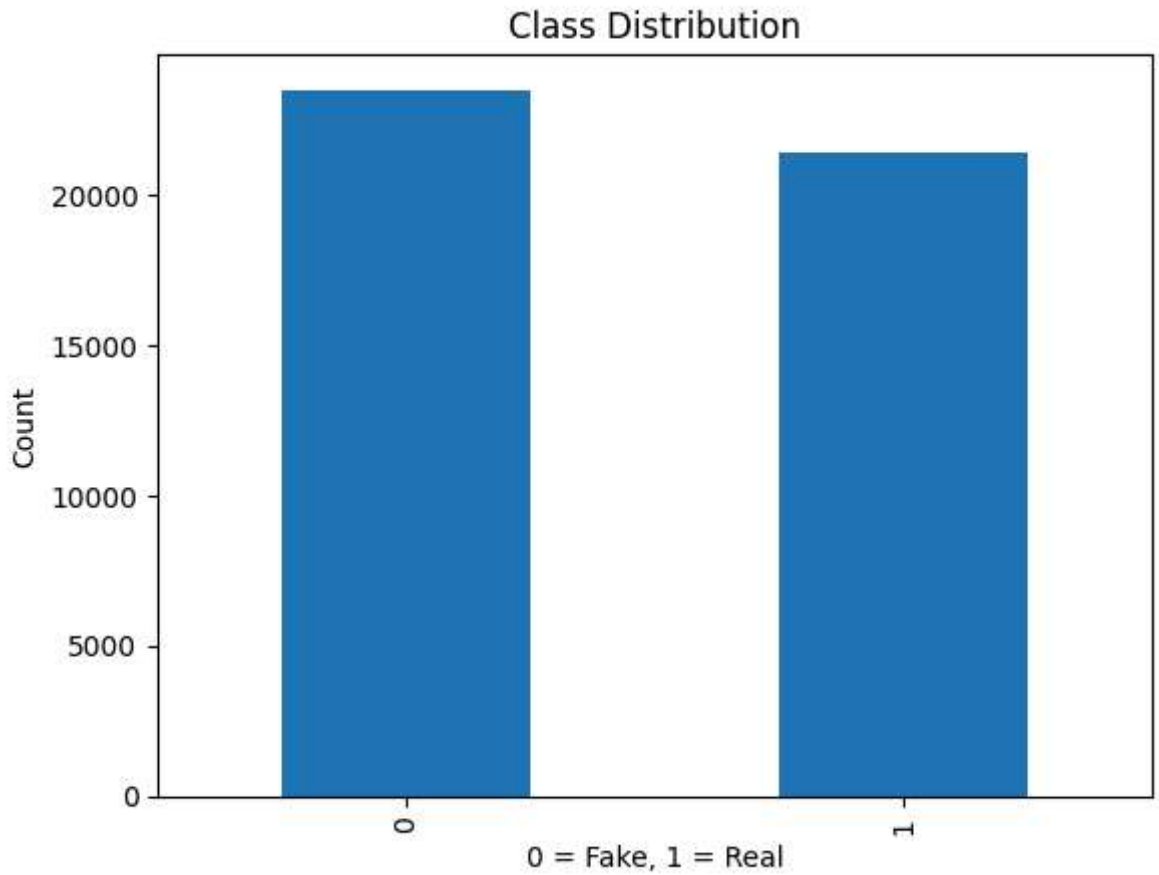

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, pred_tfidf)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Confusion Matrix (TF-IDF)")
plt.show()
```




```
df['label'].value_counts().plot(kind='bar')
plt.title("Class Distribution")
plt.xlabel("0 = Fake, 1 = Real")
plt.ylabel("Count")
plt.show()
```

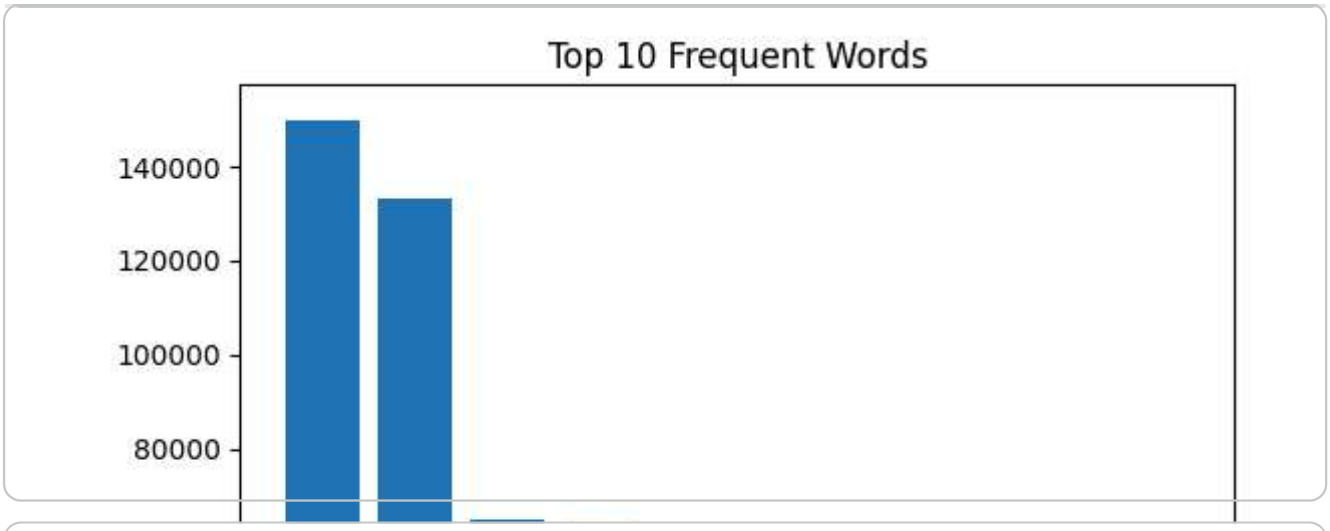


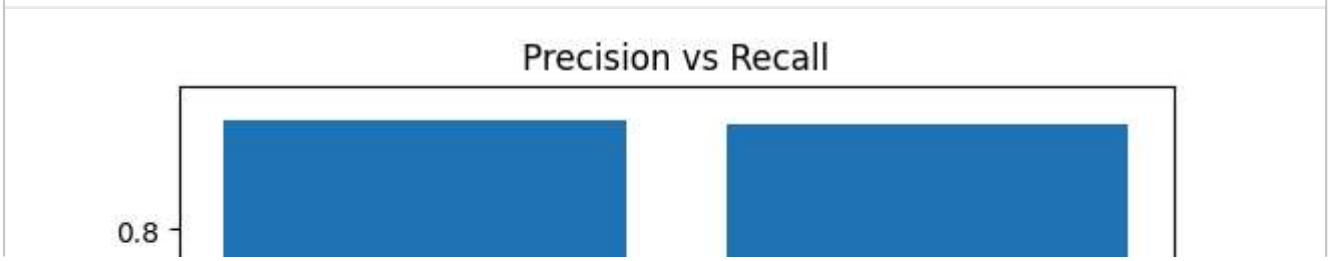

```
from collections import Counter

all_words = " ".join(df['stemmed_text']).split()
common_words = Counter(all_words).most_common(10)

words = [w[0] for w in common_words]
counts = [w[1] for w in common_words]

plt.figure()
plt.bar(words, counts)
plt.title("Top 10 Frequent Words")
plt.xticks(rotation=45)
plt.show()
```




```
from sklearn.metrics import precision_score, recall_score

precision = precision_score(y_test, pred_tfidf)
recall = recall_score(y_test, pred_tfidf)

metrics = ['Precision', 'Recall']
values = [precision, recall]

plt.figure()
plt.bar(metrics, values)
plt.title("Precision vs Recall")
plt.show()
```